结论（一页）

> 白酒行业财报 RAG 问答知识库 · 10 题评测结论 真值全部逐条从年报原文核对（见 `真值核对.md`），**不凭记忆**

一、实验设置

- 语料：白酒行业 **12 家公司 × 2024 年报**，共 **12 份 PDF、2235 页**（巨潮资讯网，`manifest.csv` 留有来源 URL 与 sha256）
- 抽取：文字块 **2235** 个、表格块 **3777** 个（共 6012），表格按行列还原成 Markdown
- 切块：**7504** 个 chunk（文本 3727 / 表格 3777），平均 **506 字**（中位 455，最长 4951），**78.6% 含数字**
- 检索：向量 **bge-m3** + **FAISS** + **BM25（jieba + 财报自定义词典）**，RRF 融合（k=60），各召回 top-20 → 融合取 top-6
- 全景题：走**跨公司配额召回**（12 家 × 3 块，轮询交错后取 36 块）
- 生成模型：**Qwen2.5-1.5B-Instruct**（本地 transformers，fp16，RTX 3050 4GB）
- 评测：**10 题**，其中跨公司全景题 3 道

二、总体结果

题型	题数	正确	数字对但口径错	错误	拒答	准确率
单公司 · 单指标	3	**3**	0	0	0	**100%**
单公司 · 表格多指标	2	1	1	0	0	50%
单公司 · 跨年度对比	2	1	0	1	0	50%
跨公司 • 全景/比较	3	**0**	0	**3**	0	**0%**
合计	**10**	**6**	**1**	**3**	**0**	**60%**

> 准确率按严格口径：**「数字对但口径错」不算正确**。若把它计入，则为 70%。

三、答得好的题，为什么好

Q01–Q04 全对，且都定位到正确页码。 原因有三，都是设计里明确的取舍：

1. **指标名是年报固定用语**（「营业收入」「归属于上市公司股东的净利润」「经营活动产生的现金流量净额」），BM25 用财报自定义词典分词后能精确命中；
2. **块首注入上下文头** `【贵州茅台 600519 · 2024年报 · 第三节 · P23】`，公司过滤生效，不会串到同行业其他公司；
3. **表格按行列还原成 Markdown**——这是最关键的一条。Q04 的分产品营收 + 毛利率来自同一张表，若表格被压成一行乱序文字，「茅台酒 1459.28 亿 / 94.06%」和「系列酒 246.84 亿 / 79.87%」的对应关系必然丢失。

具体表现：Q01（茅台营收 + 同比）、Q02（泸州老窖归母净利）、Q03（汾酒经营现金流）、Q04（茅台分产品）四项数值与出处全部正确。 归因：**单公司 × 单指标是这套架构的能力区**——检索精度高、上下文短、模型只需做「读数」而不必做推理。

四、翻车的题，错在哪

题号	问题	错因（固定枚举）	具体表现
Q05	洋河分地区营收及占比	`单位口径混淆`	数字对（省内 130.32 亿），但写成「13,031,872,833.19 **万元**」——来源块明明白白标着「单位=元」。数值正确、单位错标 1 万倍
Q07	古井贡酒两期销售费用率	`数字算错`	直接把表里的「同比增减 +13.70%」当成了销售费用率。真值 26.22%（2024）/ 26.84%（2023），是**微降**，它答「呈上升趋势」
Q08	哪家毛利率最高	`跨公司未聚合`	36 块**已覆盖全部 12 家**，模型没做比较，只倾倒了一段洋河的分产品原文
Q09	哪些公司营收下滑	`跨公司未聚合`	倾倒舍得酒业表格原文，未给结论
Q10	12 家营收合计	`跨公司未聚合`	反复输出「餐饮业 / 酒类」表头，未求和

五、错误归因统计

错因	次数	占比
未召回	0	0%
召回但被忽略	0	0%
表格切碎	0	0%
单位口径混淆	**1**	20%
跨公司未聚合	**3**	60%
检索被单公司霸榜	0	0%
数字算错	**1**	20%

> **一句话结论**：错误集中在**跨公司未聚合**（3/5），瓶颈在**生成端的多源数值聚合**，而**不是检索覆盖**。
>
> 这个判断有硬证据：三模式对比（见下节）显示，3 道全景题的召回**已经覆盖全部 12 家**， 配额召回完全生效—— `检索被单公司霸榜` 一项是 **0**。 也就是说：**该找的都找到了，模型没算。** 1.5B 模型在 36 块长上下文下直接放弃归纳、退化成复读原文。

六、三种检索模式对比（加分项）

同一批 10 题分别在三种模式下只跑检索（`tools/mode_compare.py` → `eval/mode_compare.md`）：

检索模式	单公司题命中	全景题覆盖	全景题召回块数	备注
仅向量	7/7	12/12 家	36	语义泛化好
仅 BM25	**6/7**	12/12 家	**27**	Q01 漏掉茅台营收；全景题块数不足
混合（RRF）	**7/7**	12/12 家	36	综合最优

两个值得记的观察：

1. **仅 BM25 在全景题上只召回 27 块（<36）**——12 家里有几家一个关键词都没匹配上。 这正是混合检索的意义：单一通路会漏公司，RRF 融合把覆盖补齐。
2. 三种模式的全景题召回块有实质差异（仅向量 vs 混合交集仅 29/43）， 在页面上切换模式**确实会换一组证据**，不是摆设。

七、改进方向（按性价比排序）

1. **补数值聚合层**（对全景题提升最大）：检索后先让模型「按公司抽数值 → 输出小表格 → 再比较/求和」， 而不是把 36 块长文本直接丢给它算。当前失败的三题全在这里。
2. **表格结构化存储**：三张主表另存为 dict，数值题走 dict 精确查询，不走文本检索——可同时消灭 Q05 的单位错标。
3. **单位写进片段正文**：Q05 的来源块元数据里有 `单位=元`，但正文数字旁没有，模型只能猜。把单位直接缀在数字后（`13,031,872,833.19 元`）成本最低、收益直接。
4. **加 reranker**：bge-reranker 对 top-20 重排，减少「召回但被忽略」。
5. **上更大的生成模型**：1.5B 是本次最大的单点瓶颈，换 7B 级（ollama / API）预计全景题会明显改善——可作为对照实验。

八、交付清单自查

- [x] 代码仓库（含 README、可复现步骤）
- [ ] 页面截图（问答结果 + 引用来源 + 检索过程）
- [x] 本页结论

> 截图说明：本地 `streamlit run app/streamlit_app.py` → `http://localhost:8501`， 在输入框问 Q01/Q08 等题，展开「引用来源」与「检索过程」后截图即可。